

Inventario de Endpoints — API REST CommerCity

Fecha: 2026-09-05 (versión sobre el inventario del 2026-08-28) Autor: Daniel Palacios (líder backend)
Versión: 1.1 (para el grupo API REST; verificación QA y E2E del 2026-09-05)

Convenciones generales

- **Base URL:** `http://localhost:3000` (desarrollo; puerto real del backend, ver `backend/src/server/server.js`). Producción según despliegue.
- **Base de datos (desarrollo):** `commercity_v2` en `149.130.178.228:3306`, usuario `commercity_user` (verificado E2E el 2026-09-05). No usar `commercity_v2` como nombre de BD en el `.env`.
- **Prefijo:** todos los endpoints bajo `/api`.
- **Autenticación:** `Authorization: Bearer <token>` (JWT). `401` sin token, `403` rol insuficiente.
- **Respuesta estándar:**
- **Éxito:** `{ "success": true, "data": ... }`
- **Error:** `{ "success": false, "error": { "code": "UNAUTHORIZED|FORBIDDEN|NOT_FOUND|VALIDATION_ERROR|INTERNAL_ERROR", "message": "..." } }`
- **Subida de archivos:** campos multipart `archivo` (chat), `imagen` (productos) y `evidencia` (reportes). Archivos servidos desde `/uploads`.
- **Moneda:** COP (`DECIMAL(10,2)`). El precio publicado incluye IVA 19%: `subtotal = precio / 1.19`; comisiones 90/10 sobre el subtotal.
- **Paginación:** coexisten dos contratos según el módulo (catálogo vs Mi Tienda), ver sección 13.

0. Health check (raíz)

Método	Ruta	Acceso	Recibe	Devuelve
GET	<code>/</code>	Público	-	<code>{ success, data }</code> listado básico (sirve de health check del API)

1. Autenticación y usuarios — /api/usuarios

Método	Ruta	Acceso	Recibe	Devuelve
GET	/	Público	-	{ success, data } listado básico
POST	/register	Público	{ nombre_completo, email, password }	201 { success, data: { usuario, token } } (rol NO se envía; se asigna "comprador" por defecto)
POST	/login	Público	{ email, password }	200 { success, data: { token, usuario } }
POST	/recover	Público	{ email }	200 { success } (link expira 5 min, RF4)
POST	/reset-password	Público	{ token, nueva_password }	200 { success }
POST	/logout	JWT	-	200 { success } (revoca token)
GET	/me	JWT	-	{ success, data: perfil }
PATCH	/me/rol	JWT	{ rol: "comprador" "vendedor" }	200 { success, data }
GET	/perfil-publico/:id	Público	param id	{ success, data } perfil sin email (RF110)
DELETE	/cuenta	JWT	-	200 { success } (borrado lógico RF40)

Método	Ruta	Acceso	Recibe	Devuelve
GET	<code>/admin</code>	Admin	-	<code>{ success, data }</code> datos admin

2. Carrito — `/api/carrito`

Método	Ruta	Acceso	Recibe	Devuelve
POST	<code>/</code>	JWT	<code>{ producto_id, cantidad }</code>	<code>201 { success, data }</code>
GET	<code>/</code>	JWT	-	<code>{ success, data }</code> agrupado por vendedor con resumen (IVA/ 90-10 en vuelo)
PATCH	<code>/:productoId</code>	JWT	<code>{ cantidad }</code>	<code>200 { success, data }</code>
DELETE	<code>/:productoId</code>	JWT	-	<code>200 { success }</code>

3. Pedidos y pago — `/api/pedidos`

Método	Ruta	Acceso	Recibe	Devuelve
GET	<code>/resumen</code>	JWT	-	<code>{ success, data }</code> resumen carrito con desglose
POST	<code>/confirmar-pago</code>	JWT	<code>{ direccion_envio, metodo_pago, numero_tarjeta?, nombre_tarjeta? }</code>	<code>201 { success, data }</code> (transacción ACID: pedido+stock+pago)
PATCH	<code>/:id/estado</code>	Vendedor/Admin	<code>{ estado: "En camino" "Entregado" }</code>	<code>200 { success, data }</code> avance de estado por línea

4. Historial de compras — `/api/historial`

Método	Ruta	Acceso	Recibe	Devuelve
GET	<code>/compras</code>	JWT	query <code>?estado=</code>	<code>{ success, data }</code> historial agrupado con IVA 19% (RF26-RF32)
POST	<code>/compras/:id/cancelar</code>	JWT	-	<code>200 { success, data }</code> cancela línea Pendiente, restituye stock y reembolsa (RF35)

5. Catálogo / Panel Principal (router de productos montado en `/api`)

Rutas completas. El listado es `GET /api/productos`. Las categorías y vendedores viven en `GET /api/categorias` y `GET /api/vendedores` (NO bajo `/api/productos/categorias`, que no existe como ruta propia).

Método	Ruta	Acceso	Recibe	Devuelve
GET	<code>/api/productos</code>	Público	query <code>? page=&limit=&nombre=&categoria=&vendedor=</code>	<code>{ success, data }</code> paginado, default <code>limit=4</code> , máx 100 (RF87-RF94)
GET	<code>/api/categorias</code>	Público	-	<code>{ success, data }</code>
GET	<code>/api/vendedores</code>	Público	-	<code>{ success, data }</code>
GET	<code>/api/productos/mis-productos</code>	Vendedor	-	<code>{ success, data }</code> (RF45-RF49)
GET	<code>/api/productos/:id</code>	Público	param <code>id</code>	<code>{ success, data }</code> detalle con vendedor (RF78/RF79)

Método	Ruta	Acceso	Recibe	Devuelve
GET	/api/productos/:id/ validar-stock	Público	param <code>id</code> + query <code>?cantidad=</code>	{ success, data } (RF86)
POST	/api/productos	Vendedor	multipart: campos + <code>imagen</code>	201 { success, data } (RF44)
PUT	/api/productos/:id	Vendedor	multipart: campos + <code>imagen</code>	200 { success, data }

6. Mi Tienda — /api/tienda

Método	Ruta	Acceso	Recibe	Devuelve
GET	/mi-cuenta-bancaria	Vendedor	-	{ success, data } cuenta completa (RNF11 cifrado)
GET	/mi-cuenta-bancaria/ masked	Vendedor	-	{ success, data } cuenta enmascarada (RF122)
POST/ PUT	/mi-cuenta-bancaria	Vendedor	{ titular_nombre, banco, tipo_cuenta, numero_cuenta }	201/200 { success, data } upsert transaccional
GET	/ventas	Vendedor	query ? estado=&fecha_de sde=&fecha_hasta =&q=&pagina=	{ success, data } historial ventas + resumen 90/10 (RF129 oculta canceladas)
GET	/ingresos	Vendedor	query ? fecha_desde=&fec ha_hasta=&agrupa r_por=	{ success, data } ingresos 90% + consistencia
GET	/dashboard/stats	Vendedor	-	{ success, data } tarjetas, por estado, 6 meses

Método	Ruta	Acceso	Recibe	Devuelve
GET	/validacion	Vendedor	-	{ success, data } validación RF130-RF139 (cuenta, 90/10, devoluciones)

7. Chat — /api/chat

Método	Ruta	Acceso	Recibe	Devuelve
POST	/	JWT	multipart: receptor_id , mensaje , archivo?	201 { success, data: { id } } (RF105)
GET	/conversaciones	JWT	-	{ success, data } lista con último mensaje y no leídos
GET	/mensajes/:usuarioId	JWT	param usuarioId	{ success, data } historial con el usuario
PATCH	/mensajes/:id/leido	JWT	param id	200 { success, data }

8. Notificaciones — /api/notificaciones

Método	Ruta	Acceso	Recibe	Devuelve
GET	/	JWT	query ? tipo=&limite=	{ success, data } listado (RF100/ RF101)
GET	/no-leidas	JWT	-	{ success, data } contador (RF104)

Método	Ruta	Acceso	Recibe	Devuelve
PATCH	/leidas	JWT	-	200 { success, data }
PATCH	/:id/leida	JWT	param id	200 { success, data }
DELETE	/	JWT	-	200 { success } limpiar todas (RF102)
DELETE	/:id	JWT	param id	200 { success } (RF102)

9. Calificaciones — /api/calificaciones

Método	Ruta	Acceso	Recibe	Devuelve
POST	/vendedor	JWT	{ pedido_id, vendedor_id, estrellas: 1-5 }	201 { success, data } (RF107)

10. Seguidores — /api/seguidores

Método	Ruta	Acceso	Recibe	Devuelve
POST	/	JWT	{ usuario_id }	201 { success, data } (RF106)
DELETE	/:id	JWT	param id	200 { success }
GET	/siguiendo	JWT	-	{ success, data }
GET	/seguidores	JWT	-	{ success, data }

11. Reportes — `/api/reportes`

Método	Ruta	Acceso	Recibe	Devuelve
POST	<code>/</code>	JWT	multipart: <code>tipo_reporte</code> , <code>producto_id?</code> , <code>usuario_reportado_id?</code> , <code>motivo</code> , <code>evidencia?</code>	<code>201 { success, data }</code> (RF62/RF63/RF79/RF101)

12. Panel Admin — `/api/admin` (todas exigen rol administrador)

Método	Ruta	Recibe	Devuelve
GET	<code>/stats</code>	-	<code>{ success, data }</code> estadísticas (RF55-RF59)
GET	<code>/usuarios</code>	-	<code>{ success, data }</code>
PATCH	<code>/usuarios/:id/estado</code>	<code>{ estado }</code>	<code>200 { success, data }</code>
DELETE	<code>/usuarios/:id</code>	-	<code>200 { success }</code>
GET	<code>/productos</code>	-	<code>{ success, data }</code>
DELETE	<code>/productos/:id</code>	-	<code>200 { success }</code> (soft)
PATCH	<code>/productos/:id/restaurantar</code>	-	<code>200 { success, data }</code>
GET	<code>/reportes</code>	query <code>?q=&estado=</code>	<code>{ success, data }</code> (RF63-RF69)
GET	<code>/reportes/:id</code>	-	<code>{ success, data }</code>
DELETE	<code>/reportes/:id</code>	-	<code>200 { success }</code> (archivado)
PATCH	<code>/reportes/:id/resolver</code>	<code>{ respuesta }</code>	<code>200 { success, data }</code>
GET	<code>/busqueda</code>	query <code>?q=</code>	<code>{ success, data }</code>

Método	Ruta	Recibe	Devuelve
GET/ POST/ PUT	<code>/mi-cuenta-bancaria</code>	igual que tienda	cuenta de CommerCity (RF75/ RF76)

13. Contrato de paginación

Existen dos contratos distintos según el módulo (implementados en `productos.controllers.js` y `tienda.controllers.js`):

- **Catálogo de productos** — `GET /api/productos` : query `?page=` y `?limit=` (default `limit=4` , máx 100; filtros opcionales `nombre` , `categoria` , `vendedor`). Respuesta: `{ success, data: { pagina, limite, totalProductos, totalPaginas, hayPaginaAnterior, hayPaginaSiguiente, productos } }` .
- **Mi Tienda** — `GET /api/tienda/ventas` y `GET /api/tienda/ingresos` : query `?pagina=` (alias `page`) y `?por_pagina=` (alias `limit`), default `10` , máx 100. Respuesta: `{ success, data: { ..., total_registros, total_paginas, pagina_actual, registros_por_pagina } }` .
- **Notificaciones** — `GET /api/notificaciones` : solo `?limite=` (default 50, máx 100); sin paginación por páginas.

14. Estado de validación (2026-09-05)

- **Conexión BD real resuelta**: la base correcta es `commercity_v2` (usuario `commercity_user`); con `DB_NAME` erróneo (`commercy_v2`) MySQL devuelve 500 Access denied. Verificado con el backend en `http://localhost:3000` .
- **E2E catálogo (RF87-RF94/RF78/RF79/RF86) verificado contra BD real**: `GET /api/productos` 200 con 310 productos; `?limit=200` fuerza `limite=100` ; página 2 con `totalPaginas=78` ; detalle de producto 585 200 con vendedor y precio; `validar-stock` 585 200 `{ valido:true, stock_disponible:7, estado:"Disponible" }` ; `GET /api/categorias` 17; `GET /api/vendedores` 17.
- **Contrato de error verificado**: 400 en ID no numérico, 401 sin token y 404 NOT_FOUND estructurado en producto inexistente.
- **QA backend**: suite completa 307/307 (20 archivos); se persistió `testTimeout: 20000` en `backend/package.json` porque el default de 5s producía timeouts ambientales en corridas completas.

Resumen

- **Módulos**: 14 (health check, usuarios, carrito, pedidos, historial, productos, tienda, chat, notificaciones, calificaciones, seguidores, reportes, admin + router base).

- **Total endpoints:** 69.
- **Verificación (2026-09-05):** suite completa 307/307 tests (20 archivos), cobertura Lines 93,72% (umbral mínimo 60%).
- **Rama oficial:** commercycity, rama `backend` (commit `eaf9c91`, al día al 2026-09-05). Pendiente versionar en esa rama: migraciones SQL 008-011, `backend/.env.example` y el `testTimeout` de `backend/package.json`.